Semana 10: Machine Learning clásico — regresión, clasificación y scikit-learn

Nivel: Intermedio | **Prerrequisitos:** Semana 9 (NumPy, Pandas, Matplotlib), Semana 8 (probabilidad, media, desviación, normalización, semilla reproducible), Semana 7 (vectores, producto punto, derivada y descenso del gradiente), Semana 6 (clases con `.fit()` / `.predict()`, pruebas con `pytest`) y Semana 4 (listas y tuplas) | **Versión:** v1.0 | **Fecha:** 2026-10-26

Objetivos de aprendizaje

1. Explicar qué es el **aprendizaje supervisado** y distinguir **características** (`X`) de **etiqueta** (`y`).
2. Distinguir un problema de **regresión** (predecir un número) de uno de **clasificación** (predecir una categoría).
3. Entrenar y usar un modelo de scikit-learn con el patrón `fit` / `predict` / `score`.
4. **Dividir los datos** en entrenamiento y prueba, y explicar por qué evaluar con datos que el modelo nunca vio.
5. Medir un modelo con métricas básicas (error cuadrático medio, exactitud, matriz de confusión) e identificar **sobreajuste**.
6. Encadenar preprocesamiento y modelo en un `Pipeline` para evitar fugas de datos.

---

1. Qué es el Machine Learning clásico

Hasta ahora tú escribías las reglas: "si llueve, lleva paraguas". En Machine Learning haces lo contrario: le das al computador **ejemplos con su respuesta** y él **descubre la regla**.

Analogía: estudiar con exámenes de años anteriores. Un estudiante resuelve muchos exámenes viejos con las respuestas a la vista y poco a poco capta el patrón. El día del examen real (preguntas que nunca vio) se nota si aprendió el patrón o si solo memorizó respuestas. Eso es el **aprendizaje supervisado**: aprender de ejemplos ya resueltos.

TÉRMINO	SIGNIFICADO	EJEMPLO (PRECIO DE CASAS)
Característica (feature)	Dato de entrada; una columna de `X`	metros cuadrados, número de cuartos
Etiqueta (target)	La respuesta que queremos predecir; `y`	el precio
Ejemplo	Una fila de `X` con su `y`	una casa concreta
Entrenar (fit)	Ajustar el modelo a los ejemplos	calcular la mejor recta
Predecir (predict)	Usar el modelo con datos nuevos	estimar el precio de una casa no vista

`X` es una matriz de `n_ejemplos × n_características` (la matriz de la Semana 7, el arreglo de NumPy de la Semana 9) e `y` es un vector con un valor por ejemplo.

Hay dos grandes tipos de problema supervisado:

- **Regresión:** la respuesta es un **número** continuo (precio, temperatura, minutos de retraso).
- **Clasificación:** la respuesta es una **categoría** (spam o no spam, especie de flor, aprobado o reprobado).

> Existe también el aprendizaje no supervisado (sin etiquetas, p. ej. agrupar clientes parecidos). No se cubre en esta clase; aquí nos concentramos en el supervisado, que es la base.

---

2. Regresión lineal: ajustar una recta

En la Semana 7 una neurona aprendía un peso `w` para predecir `y ≈ w·x`. La **regresión lineal** es la misma idea con un término extra:

```
y ≈ w · x + b
```

`w` (el **coeficiente**) dice cuánto sube `y` por cada unidad de `x`; `b` (el **intercepto**) es el valor base. Con varias características, `w` es un vector y `w · x` es el producto punto de la Semana 7.

¿Cómo se elige la "mejor" recta? Midiendo el error de cada predicción y minimizando su promedio: el **error cuadrático medio** (MSE), `promedio((y_real - y_predicho)²)`. Es el "error" que el descenso del gradiente de la Semana 7 reducía; en regresión lineal existe además una fórmula cerrada, y scikit-learn la usa.

```
import numpy as np
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(42)                                  # semilla (Semana 8)
m2 = rng.uniform(30, 120, size=100)                              # metros cuadrados
precio = 1500 * m2 + 20000 + rng.normal(0, 8000, size=100)       # relación real + ruido

X = m2.reshape(-1, 1)        # scikit-learn exige X en 2D: (ejemplos, características)
modelo = LinearRegression().fit(X, precio)

print(modelo.coef_)          # [1506.8…] → ≈ 1500 $ por m² (el valor real era 1500)
print(modelo.intercept_)     # 19385.2…  → ≈ 20000 (el valor real era 20000)
print(modelo.predict([[80]]))  # [139930.1…] → precio estimado de una casa de 80 m²
```

El modelo **recuperó** casi exactamente la relación escondida en los datos (1500 y 20000), a pesar del ruido. Eso es "aprender".

---

3. Clasificación: predecir una categoría

Para clasificar, la respuesta no es un número libre sino una **clase**. El modelo más usado como punto de partida es la **regresión logística**. A pesar del nombre, es un clasificador: calcula una puntuación lineal (`w · x + b`) y la convierte en una **probabilidad** entre 0 y 1 (Semana 8). Si la probabilidad de "spam" es 0,87, el modelo predice "spam".

Un segundo modelo, muy intuitivo, es el **árbol de decisión**: una cadena de preguntas de sí/no ("¿el pétalo mide menos de 2,5 cm?") hasta llegar a una respuesta, como el juego de las veinte preguntas.

Ambos comparten la misma interfaz, por eso cambiar de modelo cuesta una línea:

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

modelo = LogisticRegression(max_iter=1000)   # o DecisionTreeClassifier(random_state=42)
modelo.fit(X_entrenamiento, y_entrenamiento)  # aprender
modelo.predict(X_nuevos)                      # clase para cada fila
modelo.predict_proba(X_nuevos)                # probabilidad de cada clase
modelo.score(X_prueba, y_prueba)              # exactitud (accuracy)
```

Esto es POO de la Semana 6: un objeto con estado interno (lo aprendido) y métodos `fit` / `predict`. En scikit-learn se llaman **estimadores**.

---

4. Entrenamiento, prueba y sobreajuste

Si evalúas con los mismos ejemplos con los que entrenaste, es como calificar al estudiante con el examen que ya memorizó: la nota no dice nada. Por eso **se separan los datos**:

- **Entrenamiento (≈ 80 %):** con esto el modelo aprende.
- **Prueba (≈ 20 %):** se guarda bajo llave; solo se usa al final para medir cómo se comportará con datos nuevos.

```
from sklearn.model_selection import train_test_split

X_ent, X_prueba, y_ent, y_prueba = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y   # stratify: misma proporción de
clases
)
```

`random_state=42` fija la semilla de la Semana 8: la división es reproducible. `stratify=y` (solo en clasificación) mantiene la proporción de cada clase en ambos grupos.

Sobreajuste (overfitting): el modelo "memoriza" el entrenamiento, incluido el ruido, y falla con datos nuevos. Señal típica: **muy buena nota en entrenamiento y notablemente peor en prueba.** Un árbol sin límite de profundidad puede memorizar todo: obtiene 1,0 en entrenamiento. Lo opuesto es el **subajuste**: un modelo demasiado simple que no capta el patrón ni en entrenamiento.

Métricas básicas

PROBLEMA	MÉTRICA	QUÉ MIDE
Regresión	**MSE** / **RMSE** (raíz del MSE, en las unidades de `y`)	Error promedio de las predicciones; menor es mejor
Regresión	**R^2**	Fracción de la variación de `y` que el modelo explica (1,0 = perfecto)
Clasificación	**Exactitud** (accuracy)	Porcentaje de aciertos
Clasificación	**Matriz de confusión**	Tabla de qué clase real se confundió con cuál

> **Cuidado con la exactitud sola:** si solo el 1 % de los correos son fraude, un modelo que siempre dice "no es fraude" acierta el 99 % y no sirve de nada. Con clases desbalanceadas se miran también precisión y exhaustividad (precision / recall), que veremos más adelante.

---

5. `Pipeline`: preprocesar y modelar sin trampas

En la Semana 8 normalizaste datos con `(v - v.mean()) / v.std()`. En scikit-learn eso es `StandardScaler`. Hay una trampa: la media y la desviación deben calcularse **solo con el entrenamiento**; si usas también los datos de prueba, "filtras" información del futuro al modelo (**fuga de datos**, data leakage) y tu métrica sale falsamente optimista.

El `Pipeline` encadena pasos y garantiza el orden correcto: al llamar `fit`, aprende todo solo del entrenamiento; al llamar `predict`, aplica lo aprendido.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

modelo = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
modelo.fit(X_ent, y_ent)       # escala y entrena, usando solo X_ent
modelo.score(X_prueba, y_prueba)
```

6. Por qué esto importa para IA/ML

- **Es el flujo de trabajo de casi todo proyecto de ML**: datos → `X` / `y` → dividir → entrenar → evaluar → predecir. Las redes neuronales de la próxima semana siguen exactamente el mismo flujo.
- **Los modelos clásicos son una línea base obligatoria.** En datos tabulares (hojas de cálculo, bases de datos) suelen igualar o superar a modelos más complejos, y son rápidos, baratos e interpretables. Antes de usar algo sofisticado, se prueba una regresión logística.
- **La regresión logística es el ladrillo del Deep Learning.** Una neurona es una puntuación lineal más una función de activación; una red es muchas de ellas apiladas.
- **La disciplina de evaluación es la misma en todo ML:** probar con datos no vistos y desconfiar de una buena nota en entrenamiento.
- **El patrón `fit` / `predict` es un estándar de facto**; muchas librerías (XGBoost, LightGBM) lo imitan.

7. Ejemplo práctico: clasificar especies de flores (Iris)

Problema: el dataset Iris trae 150 flores, cada una con 4 medidas (largo y ancho del sépalo y del pétalo, en cm) y su especie (setosa, versicolor o virginica). Entrenar un modelo que prediga la especie y medir qué tan bien lo hace con flores que no vio.

Pseudocódigo

```
cargar las 150 flores: X (medidas) e y (especie)
dividir: 80 % para entrenar, 20 % para probar (misma proporción de especies)
crear un modelo: escalar las medidas + regresión logística
entrenar el modelo solo con las flores de entrenamiento
predecir las flores de prueba
medir la exactitud y mostrar la matriz de confusión
```

Código real (Python)

```
# analisis_flores.py
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler


def cargar_datos():
    """Devuelve X (150x4) e y (150 etiquetas: 0, 1, 2) del dataset Iris."""
    return load_iris(return_X_y=True)


def dividir(X, y, semilla=42):
    """Separa 80 % para entrenar y 20 % para probar, conservando proporciones de
clases."""
    return train_test_split(X, y, test_size=0.2, random_state=semilla, stratify=y)


def crear_modelo():
    """Escalado + regresión logística en un solo objeto (evita fugas de datos)."""
    return make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))


def evaluar(modelo, X_prueba, y_prueba):
    pred = modelo.predict(X_prueba)
    return accuracy_score(y_prueba, pred), confusion_matrix(y_prueba, pred)


if __name__ == "__main__":
    X, y = cargar_datos()
    X_ent, X_prueba, y_ent, y_prueba = dividir(X, y)
    modelo = crear_modelo().fit(X_ent, y_ent)
    exactitud, matriz = evaluar(modelo, X_prueba, y_prueba)
    print(f"exactitud en prueba: {exactitud:.3f}")
    print(matriz)
```

Resultado verificado por ejecución:

```
exactitud en prueba: 0.933
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

Cómo leer la matriz: cada **fila** es la especie real y cada **columna** la predicha. La diagonal son los aciertos (28 de 30). Hay dos errores: una versicolor predicha como virginica y una virginica predicha como versicolor, especies que de verdad se parecen. La setosa se separa perfectamente.

Sobreajuste en vivo: un `DecisionTreeClassifier(random_state=42)` sin límites alcanza `1.0` de exactitud en entrenamiento y `0.933` en prueba (verificado): la nota de entrenamiento engaña, la de prueba es la que cuenta. Además, `modelo.predict_proba(X_prueba[:1])` devuelve `[[0.979 0.021 0. ]]` : el modelo está 97,9 % seguro de la primera clase; recuerda que una probabilidad es una salida, no una garantía.

Segundo ejemplo: evaluar una regresión

Con los datos de casas de la sección 2, tras dividir 80/20 (`random_state=42`) y entrenar, en el conjunto de prueba se obtuvo (verificado):

```
from sklearn.metrics import mean_squared_error, r2_score

pred = modelo.predict(X_prueba)
rmse = mean_squared_error(y_prueba, pred) ** 0.5   # ≈ 7413 → error típico de ~7 400 $
r2 = r2_score(y_prueba, pred)                       # ≈ 0.962 → explica ~96 % de la
variación
```

Un RMSE de ≈ 7 400 es coherente con el ruido que se inyectó (desviación 8 000): el modelo no puede eliminar el ruido, solo captar la relación.

Pruebas con pytest (Semana 6)

```
# test_analisis_flores.py
import numpy as np
import pytest
from sklearn.linear_model import LinearRegression
from analisis_flores import cargar_datos, crear_modelo, dividir, evaluar


def test_forma_de_los_datos():
    X, y = cargar_datos()
    assert X.shape == (150, 4)
    assert y.shape == (150,)


def test_division_80_20_conserva_clases():
    X, y = cargar_datos()
    X_ent, X_prueba, y_ent, y_prueba = dividir(X, y)
    assert len(X_ent) == 120 and len(X_prueba) == 30
    assert set(np.unique(y_prueba)) == {0, 1, 2}


def test_modelo_supera_90_por_ciento():
    X, y = cargar_datos()
    X_ent, X_prueba, y_ent, y_prueba = dividir(X, y)
    modelo = crear_modelo().fit(X_ent, y_ent)
    exactitud, matriz = evaluar(modelo, X_prueba, y_prueba)
    assert exactitud > 0.9
    assert matriz.sum() == 30


def test_probabilidades_suman_uno():
    X, y = cargar_datos()
    modelo = crear_modelo().fit(X, y)
    assert modelo.predict_proba(X[:5]).sum(axis=1) == pytest.approx(1.0)


def test_regresion_recupera_la_recta():
    X = np.arange(10).reshape(-1, 1)
    y = 3 * X.ravel() + 5
    modelo = LinearRegression().fit(X, y)
    assert modelo.coef_[0] == pytest.approx(3.0)
    assert modelo.intercept_ == pytest.approx(5.0)
```

Las cinco pruebas pasan al ejecutar `pytest` (verificado antes de publicar; versiones usadas: scikit-learn 1.9, NumPy 2.4, Pandas 3.1). Fíjate en que se prueba con umbrales (`> 0.9`) y no con igualdad exacta: un modelo de ML es una función numérica, no una receta exacta.

---

8. Notas de vigencia técnica (2026)

- **scikit-learn sigue siendo el estándar para ML clásico** sobre datos tabulares (versión 1.9 al preparar esta clase). Su API `fit` / `predict` / `transform` y los `Pipeline` son estables y se enseñan tal cual.
- **Datasets que ya no existen:** tutoriales antiguos usan `load_boston` (precios de casas de Boston); se eliminó de scikit-learn hace años por razones éticas. Usa Iris, California Housing (`fetch_california_housing`) o tus propios datos.
- **Para tablas grandes, el estándar moderno es el gradient boosting**: `HistGradientBoostingClassifier` / `Regressor` (incluidos en scikit-learn) y las librerías XGBoost, LightGBM y CatBoost. En datos tabulares suelen ser la opción más fuerte; se estudian después de dominar lo básico de esta clase.
- **Una sola división puede engañar.** En proyectos reales se usa **validación cruzada** (`cross_val_score`): varias divisiones y se promedia. Aquí se usa una división simple por claridad.
- **Reproducibilidad:** fija siempre `random_state` y las versiones de las librerías (`requirements.txt`), como en la Semana 6.
- **Pandas 3.x como entrada:** los estimadores aceptan `DataFrame` directamente; con `set_output(transform="pandas")` los transformadores devuelven tablas con nombres de columna.
- **Sobre `LogisticRegression`:** hoy `max_iter=1000` evita avisos de "no convergió" cuando hay muchas columnas sin escalar; con `Pipeline` + `StandardScaler` converge más rápido.

---

9. Errores comunes de principiante

- **Evaluar con los datos de entrenamiento** y creer que el modelo es excelente.
- **Escalar o limpiar antes de dividir** (fuga de datos); usa `Pipeline`.
- **Olvidar** `reshape(-1, 1)` cuando `X` tiene una sola característica (scikit-learn exige 2D).
- **Fiarse de la exactitud** con clases muy desbalanceadas.
- **No fijar** `random_state`**:** los resultados cambian en cada ejecución y no se pueden comparar.
- **Confundir regresión con clasificación:** elegir `LogisticRegression` para predecir un precio, o `LinearRegression` para una categoría.
- **Interpretar** `predict_proba` **como certeza** o una correlación como causalidad.
- **Complicar de entrada:** probar un modelo sofisticado sin una línea base simple con la cual compararlo.

10. Ejercicio propuesto

Con el dataset de vinos de scikit-learn (`from sklearn.datasets import load_wine`), escribe un módulo `analisis_vinos.py` con funciones `cargar_datos()`, `dividir()`, `crear_modelo()` y `evaluar()` siguiendo el mismo diseño que `analisis_flores.py`. Compara en la prueba una `LogisticRegression` con `StandardScaler` contra un `DecisionTreeClassifier` con `max_depth=3`, y muestra ambas exactitudes y matrices de confusión. Añade pruebas con `pytest` y versiona el resultado con Git en dos commits.

Pista: ¿alguno de los dos modelos muestra sobreajuste? Compara `score` en entrenamiento y en prueba y deja la conclusión como comentario.

---

11. Autoevaluación

1. ¿Qué es el aprendizaje supervisado? — Aprender una regla a partir de ejemplos que ya incluyen la respuesta correcta (`X` con su `y`).
2. ¿Regresión o clasificación? (a) predecir la temperatura de mañana; (b) decidir si un correo es spam. — (a) regresión, porque la respuesta es un número; (b) clasificación, porque es una categoría.
3. ¿Qué hacen `fit` y `predict`? — `fit` ajusta el modelo a los datos de entrenamiento; `predict` usa lo aprendido para estimar la respuesta de datos nuevos.
4. ¿Por qué no se evalúa con los datos de entrenamiento? — Porque el modelo puede haberlos memorizado; la nota no mide cómo se comportará con datos nuevos.
5. ¿Qué es el sobreajuste y cómo se detecta? — Memorizar el entrenamiento (incluido el ruido); se detecta cuando la nota en entrenamiento es muy superior a la de prueba.
6. En `y ≈ w·x + b`, ¿qué significan `w` y `b`? — `w` es cuánto cambia `y` por cada unidad de `x`; `b` es el valor base cuando `x` es 0.
7. Una matriz de confusión tiene `[[10,0,0],[0,9,1],[0,1,9]]`. ¿Cuántos aciertos hay y cuántos errores? — 28 aciertos (la diagonal) y 2 errores, sobre 30 ejemplos.
8. ¿Qué es la fuga de datos y cómo la evita un `Pipeline`? — Usar información del conjunto de prueba durante el entrenamiento o el preprocesamiento; el `Pipeline` aprende todo solo con el entrenamiento y luego aplica lo aprendido.
9. V/F: "Un modelo con 99 % de exactitud siempre es bueno." — Falso: con clases desbalanceadas, un modelo que siempre predice la clase mayoritaria obtiene esa nota sin aprender nada.

---

12. Verificación de dependencias

Depende de la Semana 9 (arreglos de NumPy como `X` / `y`, `reshape`, `DataFrame` como posible entrada, gráficos), de la Semana 8 (probabilidad en `predict_proba`, normalización que ahora hace `StandardScaler`, semilla reproducible con `random_state`, separación entrenamiento/prueba y sobreajuste, que se anticiparon allí), de la Semana 7 (producto punto en `w · x`, error a minimizar y

descenso del gradiente como idea de "aprender"), de la Semana 6 (objetos con `.fit()` / `.predict()`, pruebas con `pytest`, `pytest.approx` y versionado con Git en el ejercicio) y de las Semanas 3–4 (funciones, listas y tuplas, p. ej. el desempaquetado de `train_test_split`). scikit-learn se introduce desde cero en esta clase; ningún tema posterior (redes neuronales, PyTorch, LLMs) es necesario. Es prerrequisito directo del Módulo 11 (Deep Learning), que reutiliza el flujo `X` / `y`, la división entrenamiento/prueba y las métricas.